

- previous weeks
- * Isabelle I face
- * How to prove / state theorems in Isabelle
 - FOL
 - BQ
 - sub, t
 - Induction on nat
- * How to do structural recursion on lists
 - " mathematical induction on lists

→ This video :: algebraic data types

→ Intuition ::

- * Recall lists
- Any list can be represented as a finite number of applications of "cons"

$x_1 \text{ cons } x_2 \text{ cons } x_3 \text{ cons } \dots \text{ cons } x_n \text{ cons } ()$

- This was relevant, because we could define recursive functions on lists by considering only 2 cases / patterns

Nil ([])

Cons x xs (x cons xs)

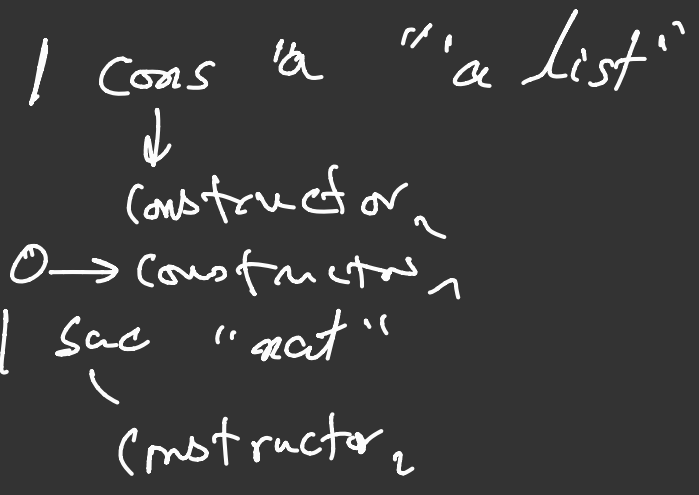
- A datatype definition formally encapsulates the intuition stated earlier
- i.e. what are the possible patterns
- & an indication of how the recursive calls on that data type progress

[1, 2, 3] 1 → 2 → 3 →

- * Naturals
- pattern
- 0
- Succ "n"

→ formally ::

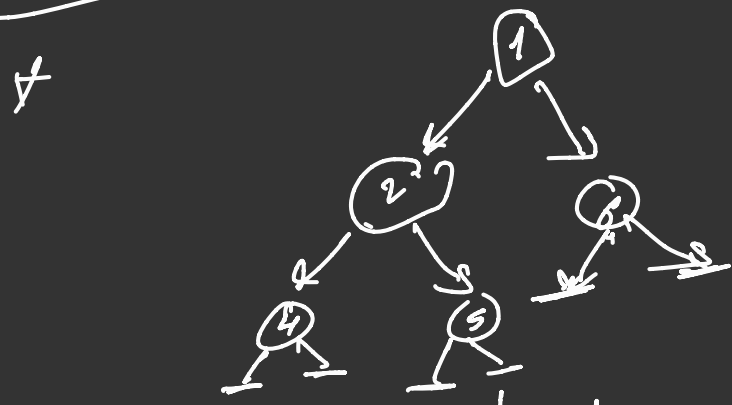
- * How do we represent the fact that a list is either Nil or Cons x xs?
- datatype 'a list = Nil | Cons x xs



* for nats ::

datatype nat = 0 | Succ "nat"

→ Binary trees

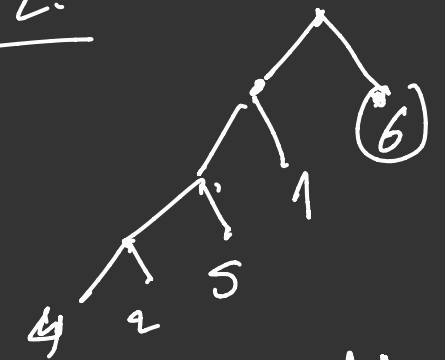


* datatype 'a tree = Leaf

| Node "a tree" 'a "a tree"

* E.g. (N (N (N L 4 4) 2 (N L 5 L)) 1 (N L 6 L))

→ Binary tree 2 ::



datatype 'a tree = Leaf 'a | Node "a tree" "a tree"